

Phase 3 API Reference

Complete API documentation for all Phase 3 features

Table of Contents

- [1. Session Management](#)
- [2. Memory System](#)
- [3. State Persistence](#)
- [4. Template System](#)
- [Error Handling](#)
- [Thread Safety](#)
- [Best Practices](#)

Session Management

Package: `dev.helix.code/internal/session`

Types

Mode

```
type Mode string
```

```

const (
    ModePlanning    Mode = "planning"    // Design and architecture
    ModeBuilding    Mode = "building"    // Implementation
    ModeTesting     Mode = "testing"     // Quality assurance
    ModeRefactoring Mode = "refactoring"  // Code improvement
    ModeDebugging   Mode = "debugging"   // Issue investigation
    ModeDeployment   Mode = "deployment"  // Release preparation
)

```

Status

```

type Status string

const (
    StatusIdle      Status = "idle"      // Created but not started
    StatusActive    Status = "active"    // Currently running
    StatusPaused    Status = "paused"    // Temporarily stopped
    StatusCompleted Status = "completed"  // Successfully finished
    StatusFailed    Status = "failed"    // Ended with failure
)

```

Session

```

type Session struct {
    ID          string          // Unique identifier
    Name        string          // Human-readable name
    Mode        Mode                // Session mode
    Status      Status            // Current status
    ProjectID   string            // Associated project
    Tags        []string          // Searchable tags
    Metadata    map[string]string // Custom metadata
    Context     map[string]string // Session context data
    CreatedAt   time.Time         // Creation timestamp
    StartedAt   time.Time         // Start timestamp
}

```

```
    EndedAt      time.Time      // End timestamp
    LastPausedAt time.Time      // Last pause timestamp
}
```

Manager

Creation

```
func NewManager() *Manager
```

Creates a new session manager.

```
mgr := session.NewManager()
```

Create Session

```
func (m *Manager) Create(name string, mode Mode, projectID string) (*Session, error)
```

Parameters:

- name: Session name
- mode: Session mode (planning, building, etc.)
- projectID: Associated project ID

```
sess, err := mgr.Create("implement-auth", session.ModeBuilding, "api-se  
if err != nil {  
    log.Fatal(err)  
}
```

Session Lifecycle

```
func (m *Manager) Start(id string) error      // idle → active
func (m *Manager) Pause(id string) error     // active → paused
func (m *Manager) Resume(id string) error    // paused → active
func (m *Manager) Complete(id string) error  // active → completed
func (m *Manager) Fail(id string, reason string) error // active → failed
```

Query Methods

```
func (m *Manager) Get(id string) (*Session, error)
func (m *Manager) GetAll() []*Session
func (m *Manager) GetByProject(projectID string) []*Session
func (m *Manager) GetByMode(mode Mode) []*Session
func (m *Manager) GetByStatus(status Status) []*Session
func (m *Manager) GetByTag(tag string) []*Session
func (m *Manager) GetRecent(limit int) []*Session
func (m *Manager) FindByName(name string) []*Session
func (m *Manager) Count() int
func (m *Manager) CountByStatus() map[Status]int
```

Export/Import

```
// Export
snapshot, err := mgr.Export(sess.ID)
data, _ := json.Marshal(snapshot)

// Import
var snapshot session.SessionSnapshot
json.Unmarshal(data, &snapshot)
mgr.Import(&snapshot)
```

Callbacks

```
mgr.OnCreate(func(s *session.Session) {
    log.Printf("Session created: %s\n", s.Name)
})
```

Memory System

Package: dev.helix.code/internal/memory

Types

Role

```
type Role string

const (
    RoleUser      Role = "user"      // User messages
    RoleAssistant Role = "assistant" // AI assistant messages
    RoleSystem     Role = "system"   // System/context messages
)
```

Message

```
type Message struct {
    ID          string          // Unique identifier
    Role        Role              // Message role
    Content     string           // Message content
    Metadata    map[string]string // Custom metadata
    Tokens      int              // Estimated token count
}
```

```
    CreatedAt time.Time // Creation timestamp
}
```

Conversation

```
type Conversation struct {
    ID          string // Unique identifier
    Title       string // Conversation title
    SessionID   string // Associated session ID
    Metadata    map[string]string // Custom metadata
    CreatedAt   time.Time // Creation timestamp
    UpdatedAt   time.Time // Last update timestamp
}
```

Message Creation

```
msg := memory.NewUserMessage("Help me debug this code")
msg.SetMetadata("file", "main.go")
```

Manager

Create Conversation

```
conv := mgr.CreateConversation("Feature Implementation")
conv.SessionID = "session-123"
```

Add Message

```
mgr.AddMessage(conv.ID, memory.NewUserMessage("Hello"))
```

Query Methods

```
func (m *Manager) GetAll() []*Conversation
func (m *Manager) GetBySession(sessionID string) []*Conversation
func (m *Manager) GetRecent(limit int) []*Conversation
func (m *Manager) Search(query string) []*Conversation
func (m *Manager) SearchMessages(query string) []*Message
func (m *Manager) TotalMessages() int
func (m *Manager) TotalTokens() int
```

Limits

```
mgr.SetMaxMessages(500)
mgr.SetMaxTokens(50000)
mgr.TrimConversations(50) // Keep last 50 conversations
```

Conversation Methods

```
// Get recent messages
recent := conv.GetRecent(10)

// Search
results := conv.Search("authentication")

// Token management
conv.SetMaxTokens(10000)
total := conv.TotalTokens()

// Export as text
text := conv.ToText()
```

State Persistence

Package: `dev.helix.code/internal/persistence`

Types

```
type Format string

const (
    FormatJSON          Format = "json"          // Human-readable JSON
    FormatCompactJSON   Format = "compact-json" // Compact JSON
    FormatJSONGZIP       Format = "json-gzip"    // Compressed JSON
)
```

Store Creation

```
store := persistence.NewStore("./helixcode_data")
```

Configuration

```
store.SetSessionManager(sessionMgr)
store.SetMemoryManager(memoryMgr)
store.SetFormat(persistence.FormatJSONGZIP)
```

Save/Load Operations

```
// Save everything
if err := store.Save(); err != nil {
    log.Fatal(err)
}
```



```
}

// Load everything
if err := store.Load(); err != nil {
    log.Println("No previous state found")
}
```

Auto-Save

```
// Enable auto-save every 5 minutes
store.EnableAutoSave(300)

// Check status
if store.IsAutoSaveEnabled() {
    interval := store.GetAutoSaveInterval()
    fmt.Printf("Auto-save every %d seconds\n", interval)
}

// Disable
store.DisableAutoSave()
```

Backup and Restore

```
// Create backup
backupPath := fmt.Sprintf("./backups/backup-%s", time.Now().Format("2006-01-02T15:04:05"))
if err := store.Backup(backupPath); err != nil {
    log.Fatal(err)
}

// Restore from backup
if err := store.Restore(backupPath); err != nil {
```

```
    log.Fatal(err)
}
```

Callbacks

```
store.OnSave(func() {
    log.Println("State saved successfully")
})

store.OnError(func(err error) {
    log.Printf("Persistence error: %v\n", err)
})
```

Template System

Package: dev.helix.code/internal/template

Types

```
type Type string

const (
    TypeCode           Type = "code"           // Code generation
    TypePrompt         Type = "prompt"         // AI prompts
    TypeWorkflow       Type = "workflow"       // Process templates
    TypeDocumentation  Type = "documentation"  // Documentation
    TypeEmail          Type = "email"          // Email templates
    TypeCustom         Type = "custom"         // Custom use
)
```

Template Creation

```
tpl := template.NewTemplate(
    "API Handler",
    "Generate HTTP handler",
    template.TypeCode,
)

tpl.SetContent(`func {{name}}() {{return_type}} {
    {{body}}
}`)

tpl.AddVariable(template.Variable{
    Name:      "name",
    Required:  true,
    Type:      "string",
})

result, err := tpl.Render(map[string]interface{}{
    "name": "ProcessData",
    "body": "return nil",
})
```

Manager

Register Template

```
if err := mgr.Register(tpl); err != nil {
    log.Fatal(err)
}
```

Render

```
result, err := mgr.RenderByName("Function", map[string]interface{}{
    "function_name": "ProcessData",
    "parameters":    "data []byte",
    "return_type":   "error",
    "body":          "return nil",
})
```

Query Methods

```
tpl, err := mgr.GetByName("Function")
codeTpls := mgr.GetByType(template.TypeCode)
results := mgr.Search("handler")
total := mgr.Count()
```

Built-in Templates

```
if err := mgr.RegisterBuiltinTemplates(); err != nil {
    log.Fatal(err)
}
```

Registers 5 built-in templates: Function, Code Review, Bug Fix, Function Documentation, Status Update Email.

Error Handling

All methods that can fail return an error type. Always check errors:

```
sess, err := sessionMgr.Create("name", session.ModeBuilding, "project")
if err != nil {
    log.Fatalf("Failed to create session: %v", err)
}
```

Common error types:

- `ErrNotFound` - Resource not found
- `ErrValidation` - Validation failed
- `ErrConflict` - Duplicate or conflicting resource
- `ErrState` - Invalid state transition

Thread Safety

All managers are thread-safe and can be used concurrently:

- Protected with `sync.RWMutex`
- Safe for concurrent reads
- Safe for concurrent writes
- No external synchronization needed

Best Practices

1. **Always check errors** - Never ignore error return values
2. **Use contexts** - Store relevant data in metadata and context fields
3. **Set limits** - Configure max messages/tokens to prevent unbounded growth
4. **Enable auto-save** - Prevent data loss with automatic persistence
5. **Clean up** - Delete or archive old sessions and conversations
6. **Use callbacks** - Monitor events for logging and analytics
7. **Tag everything** - Makes searching and filtering easier

8. **Export important work** - Backup sessions, conversations, and templates

Quick Reference

Phase 3 Features

Example Projects